The Journal of Open Source Software

uringpy: A GIL-aware io_uring runtime for CPython

Marimuthu Velayutham [1]

1 Independent Researcher

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

uringpy is a completion-reaping network runtime for CPython built on the Linux io_uring interface (Axboe, 2019). Unlike asyncio and existing io_uring-backed event loops, which dispatch every I/O completion through interpreted Python, uringpy reaps completions and drives the entire accept/receive/send protocol inside a single C region that runs with the Global Interpreter Lock (GIL) released. Because no Python object is created per completion, independent worker threads — each owning a private io_uring instance and a SO_REUSEPORT socket — run concurrently across CPU cores *within one interpreter*. The extension is built with Cython and declares free-threading compatibility, so it also runs without re-enabling the GIL on free-threaded (no-GIL) CPython builds introduced by PEP 703 (Gross, 2023).

The package ships a reproducible benchmark harness: a single-node scaling matrix, an isolated two-node driver, a response-size sweep, and a microbenchmark that isolates GIL-contention cost from system-call count. Every reported number is produced by a re-runnable script.

Statement of need

Python servers that use asyncio (Python Software Foundation, n.d.) or faster drop-in loops such as uvloop (MagicStack, n.d.) cannot scale across cores inside a single process, because the GIL serializes the per-event Python work that these loops perform on every completion. The common workaround is to run one process per core, which sacrifices shared in-process state and adds inter-process coordination.

Recent io_uring event loops for Python (uringcore contributors, n.d.; uringloop contributors, n.d.) reduce system-call overhead by reaping many completions per io_uring_enter, but they still dispatch each completion in Python and therefore inherit the same single-interpreter scaling wall. uringpy targets this gap: by keeping the completion hot path in C, it lets worker threads scale across cores in one interpreter for I/O-framing-bound services, and it provides the instrumentation needed to attribute throughput to GIL contention rather than to system-call count. It is intended for researchers and engineers studying Python concurrency, io_uring, and the ongoing GIL/free-threading transition, and as a building block for high-throughput, single-interpreter network services.

Functionality

- A Cython URingEngine exposing submission/completion primitives over liburing, a fixed-size slab allocator for I/O buffers, and a C-side reactor (serve_forever_echo) that runs the connection state machine with the GIL released.
- An optional application-handler reactor (serve_forever_app) that invokes a Python callback per request, used to characterize the boundary where per-request Python work reintroduces GIL contention.

DRAFT

- A multi-worker, `SO_REUSEPORT`-sharded server launcher supporting both thread and process models for direct comparison against `asyncio`.
- Benchmark scripts for single-node and two-node measurement, a response-size sweep, and a GIL-vs-syscall isolation probe.

Benchmarks and reproducibility

On an isolated two-node deployment, `uringpy` scales across cores within a single interpreter where `asyncio` thread-pools collapse under GIL contention, and it remains faster in absolute terms on free-threaded CPython because it performs no per-completion interpreter work. The harness also documents the honest boundaries of the approach: workloads dominated by per-request Python logic require process-level parallelism, and tiny-response throughput is network-bound. All experiments are scripted and container-based for independent re-execution.

Acknowledgements

We acknowledge the `io_uring` and `liburing` maintainers, whose interface this work builds upon.

References

Axboe, J. (2019). *Efficient IO with io_uring*. https://kernel.dk/io_uring.pdf.

Gross, S. (2023). *PEP 703: Making the global interpreter lock optional in CPython*. https://peps.python.org/pep-0703/.

MagicStack. (n.d.). *Uvloop: Ultra fast asyncio event loop*. https://github.com/MagicStack/uvloop.

Python Software Foundation. (n.d.). *Asyncio — asynchronous I/O*. https://docs.python.org/3/library/asyncio.html.

uringcore contributors. (n.d.). *Uringcore: High-performance asyncio event loop using io_uring*. https://github.com/ankitkpandey1/uringcore.

uringloop contributors. (n.d.). *Uringloop: An io_uring event loop for asyncio*. https://github.com/thearchitector/uringloop.

DRAFT